

Speaker Notes

Planar Pushing — Single-Agent vs. Self-Play
University of Groningen, FSE

Vlad Iftime s3426394

July 2026

Untitled

30-minute master thesis presentation

Topic: potential-based reward shaping for planar pushing with a UR5e robot arm

Four model variants: PPO-PBRS, PPO-Curriculum, ASP-dPose, TASP-dPose

Key result: simplest model (PPO-PBRS, 80.0%) beats every structured variant

Curriculum is competitive but adds no value; self-play degrades performance

Outline



1. **Introduction & Research Questions**
2. **Background & Related Work**
reward shaping · curriculum & self-play · push mechanics & SE(2)
3. **Problem Definition & MDP**
4. **Implementation**
four models: PPO-PBRS, PPO-Curriculum, ASP-dPose, TASP-dPose
5. **Evaluation**
30 held-out scenes · 20 trials · Isaac vs gym
6. **Results**
7. **Discussion**
8. **Conclusion & Takeaways**

3

Outline

Outline in eight parts, standard thesis structure

Background covers the five literature areas; Implementation describes the four models; Evaluation defines the protocol; Results and Discussion present and interpret the validation outcomes

One-line thesis: PBRS single-agent wins; curriculum is competitive but adds no value; self-play degrades performance



Three Research Questions

1. **RQ1:** Can potential-based reward shaping outperform hand-tuned dense rewards for goal reaching tasks in a robotic environment?
2. **RQ2:** Given a well-shaped PBRS reward function does adding a curriculum or asymmetric self-play improve final task success over a direct single-agent approach?
3. **RQ3:** Can a sufficiently informative reward (PBRS) *substitute* for an explicit curriculum in contact-rich pushing?

[Ng, Harada & Russell 1999] [Grzes 2017] [Devlin & Kudenko 2012]

4

Research Questions

RQ1: PPO with original fractional reward at 2,048 envs reaches 80%. PPO-PBRS reaches 80% at 528 envs — $4\times$ fewer. PBRS is more sample-efficient and theoretically principled .

RQ2: No ASP variant exceeds 17% across five configurations (two reward schemes, two simulators, three Alice reward structures). The structural failure mode is the perpetually stale value function under adversarial goal distribution shift.

RQ3: At sufficient batch size, PBRS substitutes for curriculum (PPO-PBRS $\approx$ PPO-Curriculum). At small batch, the curriculum helps (gym: 50% vs 10%). Substitution is regime-dependent.

Six models total: PPO-Baseline (orig. reward, 2,048 envs), PPO-PBRS, PPO-Curriculum, ASP-dPose, TASP-dPose, TASP-dPose-BP (528 envs each). ~3000 iterations per PBRS model, RTX Pro 6000.

Related Work – Reward Shaping (PBRs)



Potential-based reward shaping :

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_F$$

- **Policy invariance:** $Q'(s, a) = Q(s, a) - \Phi(s)$ – the optimal policy is unchanged for any state potential Φ .
- **Episodic validity :** $\gamma_{\text{shaping}}=1$ is safe – $\sum F = \Phi(s_T) - \Phi(s_0)$ is bounded.
- **Dynamic / arbitrary potentials** – basis for our exponential Φ .

Why it matters here: lets us design a dense, well-behaved reward *without* changing the task's optimum – the foundation of every model in this thesis.

[Ng et al. 1999] [Grzes 2017] [Devlin & Kudenko 2012] [Harutyunyan et al. 2015]

6

Related Work — Reward Shaping (PBRs)

PBRs is the theoretical foundation — Ng et al. proved you can add $\gamma\Phi(s') - \Phi(s)$ to any reward without changing what the agent learns

Policy invariance: the optimal policy is unchanged — you can design rewards purely for learning speed

Episodic validity (Grzes 2017): $\gamma_{\text{shaping}} = 1$ is safe in finite-horizon tasks — the shaping telescopes to $\Phi(s_T) - \Phi(s_0)$

Our exponential potentials $\Phi(s) = we^{-kd^2}$ build on Devlin & Kudenko's dynamic potential framework

This decoupling from policy is the core insight — reward engineering becomes a separate, principled task

Related Work – Curriculum & Asymmetric Self-Play

Forced curricula

- **Surveys**
- **Reverse / goal** : start near the goal, expand outward
- **Precision** : gradually tighten the success tolerance

Our use: a forced 3-phase pos→rot curriculum (PPO-Curriculum)

Automatic (self-play)

- **ASP** : Alice/Bob adversarial game + ABC
- **Time-based Alice** : $R_A \propto t_B - t_A$, self-regulating
- **Regret-based**

Our use: ASP-dPose (outcome) and TASP-dPose (time-based)

[Narvekar et al. 2020] [Portelas et al. 2020] [Sukhbaatar et al. 2018] [Plappert et al. 2021]

7

Related Work — Curriculum & Asymmetric Self-Play

Forced curricula: Narvekar and Portelas surveys show curriculum helps when rewards are sparse or exploration is hard

Our PPO-Curriculum uses a 3-phase pos→rot staging — position first, then progressively add rotation

Automatic curricula (self-play): ASP (Sukhbaatar, Plappert) trains Alice and Bob adversarially — Alice proposes goals, Bob solves them

Time-based Alice (Sukhbaatar 2018): $R_A \propto t_B - t_A$ creates a self-regulating curriculum where Alice is rewarded for finding goals Bob struggles with

Regret-based methods (PAIRED, AMIGo): antagonist minimizes protagonist's regret — solvable but challenging environments

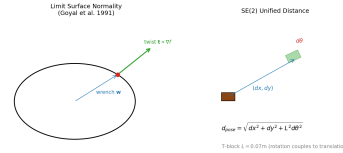
Our use: we test both outcome-based (ASP-dPose) and time-based (TASP-dPose) variants

Related Work – Geometry & Mechanics of Planar Pushing



Push mechanics

- **Quasi-static pushing:** motion set by the friction limit surface.
- **Limit-surface normality:** *every* push couples translation and rotation.



SE(2) geometry

- **Unified distance:** a single scalar that combines position and orientation error, weighted by the object's physical size L
- **Equivariant manipulation:** exploiting SE(2)/SO(2) symmetry.

Our use: the SE(2) d_{pose} provides a unified reward for the self-play variants.

[Mason 1986] [Goyal et al. 1991] [Park 1995] [Urain et al. 2023]

8

Related Work — Geometry & Mechanics of Planar Pushing

Quasi-static pushing (Mason 1986): the object motion is slow enough that inertial forces are negligible — the limit surface fully determines behavior

Limit-surface normality (Goyal 1991): the resulting motion twist $\mathbf{t}$ is always normal to the limit surface — every push couples translation AND rotation

SE(2) Lie-group distance (Park 1995): fuses position and orientation error into a single geodesic metric with characteristic length L

Equivariant manipulation: exploiting SO(2) rotational symmetry so learned pushing skills transfer across orientations

Why this matters: limit-surface coupling explains why the combined pos+rot gate is the universal bottleneck across all our models

Planar-Pushing MDP – State, Action & Transition



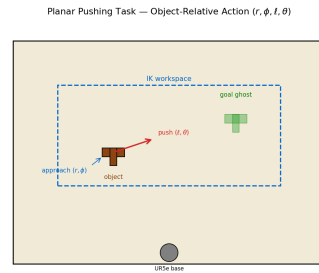
State $s \in \mathbb{R}^{28}$: EE pose (6) | object pose (14) | goal pose (6) | errors (2)

Action $a \in \mathbb{R}^4$ (object-relative): (r, ϕ, ℓ, θ) – 21 bins/dim, 194 481 discrete actions

Transition : $t \propto \nabla f(\mathbf{w})$ – limit-surface normality couples **every** push with rotation

Success: pos < 5 cm **and** rot < 0.2 rad.

Validation: 30 scenes $\times$ 20 trials $\times$ up to 30 pushes



[Mason 1986] [Lynch & Mason 1996] [Goyal et al. 1991]

10

Planar-Pushing MDP — State, Action & Transition

Planar pushing MDP: UR5e robot pushes a T-block on a 1.4m $\times$ 1.0m table. 28D obs, 4D object-relative macro-action (21 bins/dim = 194k discrete). Object-relative actions achieve 95% contact rate.

Transition: limit-surface normality couples translation and rotation — every push does both.

Success: pos < 5 cm AND rot < 0.2 rad. Validation: 30 held-out scenes $\times$ 20 trials $\times$ up to 30 pushes/trial; scene solved if any trial meets the gate.

Why the Original Reward Failed



Fractional improvement formula:

$$R = \alpha \cdot \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}} - \beta \cdot d_{\text{now}} - \beta_{\text{rot}} \cdot y_{\text{now}}, \quad \alpha=3.0, \beta=0.5, \beta_{\text{rot}}=0.25$$

Worked example – pushing toward a goal 30 cm away, 1 cm improvement:

$$R = 3.0 \cdot 0.01 / 0.30 - 0.5 \cdot 0.29 - 0.25 \cdot 0.5 = -0.17$$

Problem 1 – Negative reward for progress: the penalty terms dominate the improvement. PPO's policy gradient multiplies $\nabla \log \pi$ by $\hat{A}$ – negative advantage pushes the policy *away* from goal-directed pushes.

Problem 2 – Near-zero variance: 99% of pushes produce $R \in [-0.5, +0.1]$. Sparse bonuses (+5/+2) fire on ~1% of pushes – only ~26 bonus events per iteration at 528 envs. GAE sees noise.

PBRS contrast – same push, same 1 cm improvement:

$$R = w \cdot [e^{-k \cdot d_{\text{now}}^2} - e^{-k \cdot d_{\text{prev}}^2}] = 10 \cdot [e^{-30 \cdot 0.29^2} - e^{-30 \cdot 0.30^2}] = +0.133$$

Correctly-signed reward on every push – bounded, Markovian, policy-invariant .

11

Why the Original Reward Failed

The fractional formula gives ~0 or negative reward for 99% of pushes. A 1 cm improvement from 30 cm earns -0.17 — negative reward for genuine progress

PPO multiplies the policy gradient by the advantage sign. Negative advantage → gradient pushes policy AWAY from goal-directed behavior

Sparse bonuses (+5/+2) fire on only 1% of pushes. At 528 envs × 15 pushes = 7,920 steps/iter, that's ~79 bonus events — too few to overcome the noise

PBRS gives +0.133 for the same push — correct sign, bounded magnitude. PBRS provides a guaranteed signal floor on every push regardless of distance

Baseline – Original Reward at Scale



Same fractional reward, same PPO+LSTM, same 2,048 envs, same 100K iter budget:

Model	Scene SR	Pos-only	Pos+rot
PPO single-agent	80.0%	100%	70%
ASP (Alice+Bob)	0.0%	0%	0%

Same PPO+LSTM, same push primitive, same training budget – only the architecture differs. The fractional reward produces 80% for single-agent but 0% for ASP. The adversarial goal distribution, not the reward formula, is the bottleneck.

[Sukhbaatar et al. 2018] [Plappert et al. 2021]

13

Baseline — Original Reward at Scale

The original fractional reward at 2,048 envs reaches 80% for single-agent — the task IS solvable at scale
Same reward, same environment, same budget — ASP with Alice/Bob gets 0%. The adversarial goal distribution prevents any learning

This is the cleanest 1-axis comparison in the thesis: only the training architecture changes. Self-play is the bottleneck

All subsequent models switch to PBRS — not because the original reward is broken, but because PBRS is principled, efficient (works at 528 envs), and provides correct per-step signal even under distribution shift

PPO-PBRS – Single-Agent (Our Best Model)



Architecture: Single PPO agent, LSTM(28→256) + MLP[512, 256] trunk, MultiCategorical head (4 dims $\times$ 21 bins). No curriculum, no Alice/Bob, no GoalEncoder.

Training: 528 envs, 15 pushes/rollout, ~ 3000 iterations ~ 1.6 days wall-clock (~ 0.022 it/s).

Why it works:

1. **Dense, bounded gradient** – PBRS fires on every push
2. **Fixed goal distribution** – stationary random goals; no adversarial shift
3. **Single network** – one agent, one optimizer; no multi-agent instability

[Ng et al. 1999] [Grzes 2017] [Schulman et al. 2017]

15

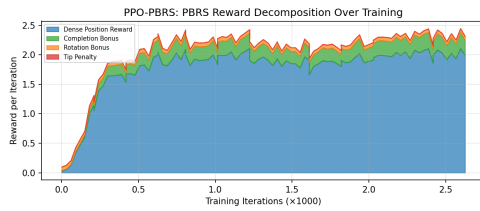
PPO-PBRS — Single-Agent (Our Best Model)

PPO-PBRS is the simplest possible architecture: single PPO agent with LSTM+MLP, PBRS reward, sparse completion bonuses

Three reasons it works: PBRS provides dense gradient at every push, fixed random goal distribution avoids adversarial shift, single network avoids multi-agent optimization instability

Converges within 1500 iterations. Stable value function bounded by exponential potentials

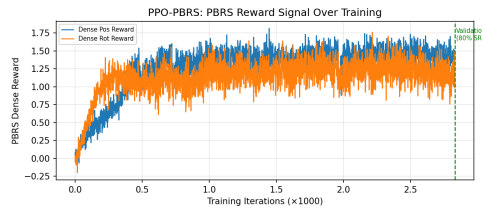
PBRS – Reward Signal Analysis



Reward diet: ~85% dense PBRS shaping,

+5/+2 sparse bonuses, penalties.

Dense signal on every push, bounded rewards, stable value – PBRS avoids the gradient starvation and critic instability of the old fractional formula.



Signal vs performance: bounded exponential potentials → stable, correlated gradient.

[Ng, Harada & Russell 1999] [Grzes 2017] [Harutyunyan et al. 2015]

16

PBRS — Reward Signal Analysis

Reward diet: ~85% of reward from dense PBRS shaping. Sparse completion bonuses (+5/+2) are supplementary — PPO always has a gradient to follow

Signal vs performance: the bounded exponential potentials produce a stable, correlated relationship between PBRS signal and training progress. The old fractional reward had near-zero variance (all pushes looked equally bad to GAE)

Cosine angular distance on rotation respects S^1 topology — no wrap-around discontinuity at $\pm\pi$

Why PBRS for ASP? – Decoupling Advantage from Value



PPO's policy gradient: $\nabla \mathcal{J} = \mathbb{E}[\nabla \log \pi(a|s) \cdot \hat{\mathbf{A}}(s, a)]$

GAE advantage estimate :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \underbrace{[r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})]}_{\delta_{t+l}}$$

Under ASP: Alice updates every iteration $\rightarrow$ Bob's goal distribution shifts $\rightarrow V(s)$ is perpetually stale. With sparse rewards ($r_t \approx 0$ for 99% of pushes):

$$\hat{A}_t \approx \sum_{l=0}^{\infty} (\gamma \lambda)^l \left[\underbrace{\gamma V(s_{t+l+1}) - V(s_{t+l})}_{\text{pure V-error - systematically biased}} \right]$$

With PBRS ($r_t = \Phi(s_{t+1}) - \Phi(s_t) + r_t^{\text{sparse}}$):

17

Why PBRS for ASP? — Decoupling Advantage from Value

PPO's gradient depends on $\hat{A}$. GAE bootstraps $\hat{A}$ from $V(s)$. If V is wrong, $\hat{A}$ is wrong, gradient is wrong.

Under ASP, Alice updates every iteration $\rightarrow$ Bob's goal distribution shifts $\rightarrow$ Bob's $V(s)$ is perpetually stale. Sparse rewards give $\hat{A} \approx$ noisy V-errors only — no ground-truth signal.

PBRS adds $\Phi(s') - \Phi(s)$ to every step. This term is ALWAYS correct — Φ increases when object moves toward goal, decreases when it moves away. Even if V is completely wrong, Bob still gets correctly-signed gradient from the Φ -component.

This is why PBRS lifts ASP from 0% to 16.7% — Bob gets genuine learning signal despite the non-stationary objective. But it only partially rescues ASP because the V -component still contributes bias.

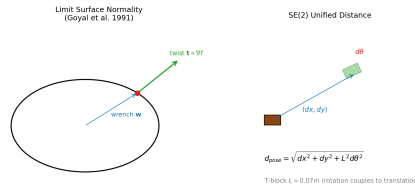
Single-agent has no distribution shift $\rightarrow V$ is accurate $\rightarrow$ both terms align $\rightarrow 80\%$.

SE(2) Unified Distance – Self-Play Variants



SE(2) metric: $d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$

- Single scalar combining (dx, dy) and $d\theta$; L = char. length (T-block 0.07 m)
- **Single-potential PBRS:** $\Phi(s) = w e^{-k d_{\text{pose}}^2}$, $k = 30$, $w = 10$
- Eliminates competing pos/rot gradients
– aligns reward with the coupled physics



Result: SE(2) does *not* rescue self-play – ASP-dPose reaches only 6.7% validation SR vs single-agent 80%. The self-play curriculum is the root cause, not the metric.

[Park 1995] [Urain et al. 2023] [Lynch & Mason 1996]

18

SE(2) Unified Distance — Self-Play Variants

The SE(2) unified distance replaces separate position and rotation errors with a single Riemannian metric on the planar rigid-body configuration space

L converts radians to meters. For the T-block with $L=0.07\text{m}$, 0.5 rad of rotation contributes 3.5cm to d_{pose} — roughly equal weight with a 3.5cm position displacement

The single-potential PBRS eliminates the competing gradient problem: separate pos/rot potentials each pull in different directions, while the SE(2) potential provides a unified gradient

Both ASP-dPose (SE(2) self-play) collapses just like the separate-potential self-play — the SE(2) metric doesn't rescue the self-play dynamics, confirming the curriculum is the root cause, not the reward metric

TASP-dPose-BobPenalty – The Fallout



Added to TASP-dPose: Bob time penalty $R_B += -\gamma_{sp} \cdot t_B$ on *all* phase ends.

Valid SR: 0.0% (0/30 scenes, 2600 iters, 528 envs) – worse than no-penalty TASP-dPose (16.7%).

Mechanism – the penalty fires on catastrophes (tip/launch/OOB):

Trajectory	PBRS Dense + Sparse	Penalty ($0.5 \times t_B$)	Total
Fail on push 1	−10.5	−0.5	−11.0
Fail on push 10	−15.0	−5.0	−20.0
Succeed push 5	+10.0	−2.5	+7.5

PPO gradient pushes Bob toward “fail on push 1” (−11.0 beats −20.0). Alice’s reward collapses: $R_A = 0.5 \cdot \max(0, 1 - 5) = 0$. **The symmetric reward creates a perverse fail-fast equilibrium.** Asymmetric (Alice incentivised, Bob unpenalised) is necessary.

[Sukhbaatar et al. 2018] [Letcher et al. 2019] [Plappert et al. 2021]

19

TASP-dPose-BobPenalty — The Fallout

Model I (TASP-dPose-BobPenalty) trains the full symmetric Sukhbaatar reward: Alice time bonus + Bob time penalty

Result: 0% SR — worse than no-penalty TASP-dPose at 16.7%. The penalty fires on ALL phase ends including catastrophes

Bob learns that failing on push 1 (−11.0) is a better return than trying for 10 pushes and failing (−20.0). PPO’s gradient pushes toward faster failure

Alice gets zero reward ($t_B \approx 1$ gives $R_A = 0$) — the self-regulating curriculum collapses on both sides

This is NOT an indictment of Sukhbaatar 2018 or Letcher 2019. It’s a reward design failure: the penalty applied to catastrophe environments creates a degenerate equilibrium

Lesson: asymmetric reward structure (Alice incentivised, Bob unpenalised) is necessary in this domain

PPO-Curriculum – Position→Rotation Curriculum

Design: Same PPO agent as PPO-PBRS + 3-phase curriculum:

1. $w_{\text{rot}} = 0$, position-only training
2. Triggered when episodic position-SR ≥ 0.5 for 20 iters: w_{rot} ramps $0 \rightarrow 10$ over 200 iters
3. Full PBRS (multi-objective)

Original trigger was mis-specified: the first version used a per-push mean position error threshold of 0.08m – below even the *best* model’s validation PosErr (0.202m). The curriculum never activated because the threshold was unreachable by construction (P82 fix).

Corrected trigger: gates on episodic position-only SR ≥ 0.5 over a 20-iter lookback. Curriculum now activates and reaches **76.7% SR** – only 3.3pp below PPO-PBRS.

[Narvekar et al. 2020] [Portelas et al. 2020] [Florensa et al. 2017]

20

PPO-Curriculum — Position→Rotation Curriculum

P82 trigger fix: old gate used per-push mean PosErr $< 0.08\text{m}$ AND over 50 iterations — unreachable. Best model’s per-push mean PosErr plateaus at $\sim 0.10\text{m}$, never below 0.08m. Fix: episodic position-only SR ≥ 0.5 over 20 iterations.

Corrected PPO-Curriculum achieves 76.7% SR, 3.3pp below PPO-PBRS. Curriculum works; doesn’t beat the simpler no-curriculum baseline.

Position precision improves from Phase 1 training (0.023m vs 0.032m); rotation slightly worse (0.663 vs 0.568 rad).

Self-Play – ASP / TASP Architecture (Alice/Bob)



Alice (Goal Proposer):

- PPO agent, free-roaming push phase – leaves T-block in a new SE(2) pose
- That pose becomes Bob's target goal
- Reward: +5 if Bob fails to reproduce it, –1 if Bob succeeds

Bob (Goal Achiever):

- PPOABC + GoalEncoder – must reproduce Alice's goal pose from scratch
- Reward: PBRS dense shaping + sparse completion bonuses (+5/+2)
- Failed trajectories stored in ABC buffer for behavioral cloning ($\beta_{abc}=0.5$)

Two variants: both use the SE(2) unified d_{pose} metric (T-block); they differ only in Alice's reward – **ASP-dPose** (outcome-based) vs **TASP-dPose** (time-based). **Excluded:** base ASP (separate pos/rot potentials) and the GoalEncoder-ablation – no matched-protocol Isaac validation.

[Plappert et al. 2021] [Sukhbaatar et al. 2018] [Torabi et al. 2018]

21

Self-Play — ASP / TASP Architecture (Alice/Bob)

The ASP architecture: Alice (goal proposer) and Bob (goal achiever) play an adversarial game

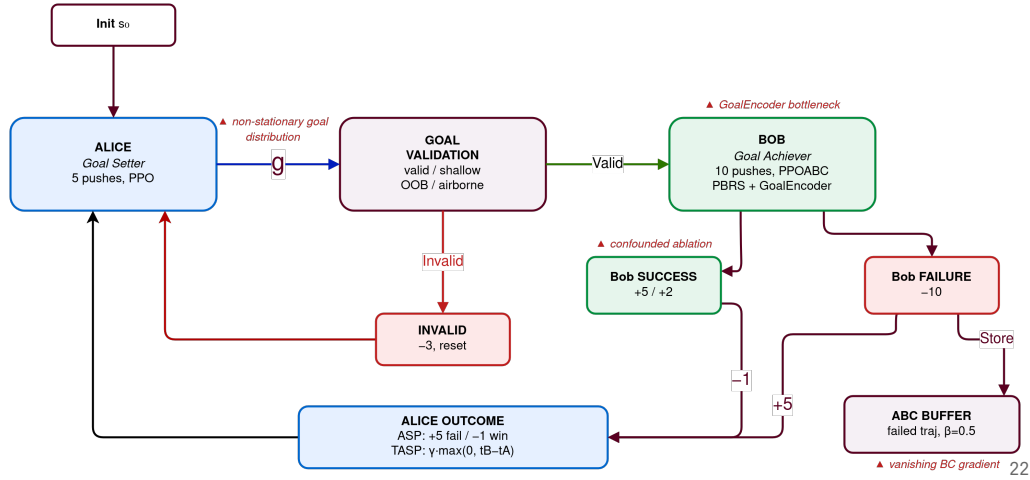
Alice gets +5 when Bob fails, –1 when Bob succeeds — incentivizing her to propose goals Bob cannot solve

Bob sees Alice's final object pose as his goal, gets PBRS dense reward + sparse completion bonuses

When Bob fails, Alice's trajectory is stored in an ABC buffer for behavioral cloning

The two self-play variants differ only in Alice's reward: ASP-dPose (outcome-based +5/–1) vs TASP-dPose (time-based, ABC disabled); both use the SE(2) d_{pose} metric on the T-block

ASP Loop – Alice↔Bob Cycle



ASP Loop — Alice↔Bob Cycle

Alice: 5-push free-roaming phase, object-relative (r, ϕ, ℓ, θ) actions, no goal observation, PPO agent. Final object pose g passed to goal validation.

Goal validation: valid (> 0.10 m displacement, $+1$), shallow (-1), OOB/airborne (-3), too-easy (-3 , reset). Invalid goals reset the episode; valid goals proceed to Bob.

Bob: starts from a clean reset at s_0 ; 10 pushes with PBRS dense shaping + sparse completion bonuses ($+5/+2$). PPOABC + GoalEncoder $\phi: \mathbb{R}^{22} \rightarrow \mathbb{R}^8$ + LSTM. Success gate: $d_{\text{pose}} < 0.055$ m (SE2) or $\text{pos} < 0.05$ m & $\text{rot} < 0.2$ rad. Failure: OOB/launched/tipped/max pushes (-10).

Non-stationary goal distribution: Alice learns continuously, so Bob’s training distribution shifts every iteration. PPO importance ratio computed from rollouts collected under a previous Alice — dominated by distribution drift, not Bob’s weight updates. (See Slides 9–9b.)

GoalEncoder bottleneck: $\phi: \mathbb{R}^{22} \rightarrow \mathbb{R}^8$ compresses object+goal into single 8D latent. Fine translation/yaw signal may be lost, preventing precise SE(2) alignment.

Vanishing BC gradient: Failed trajectories enter ABC buffer for BC ($\beta=0.5$). If Bob rarely succeeds, buffer fills with near-random trajectories — BC signal becomes noise.

Confounded ablation: Self-play vs single-agent differ on ≥ 7 axes: agent count, goal distribution, episode budget, GoalEncoder, ABC, historical pool, update frequency. $4.8\times$ SR gap cannot be attributed to any single lever (see the Caveat — Confounded Ablation slide).

ASP-dPose vs TASP-dPose – Alice Reward



Both share the SE(2) d_{pose} Bob reward and T-block object; only Alice’s reward differs.

Self-play variant	Alice reward	Valid SR
ASP-dPose (outcome-based)	$R_A = +5 \text{ fail} / -1 \text{ win}, \beta_{\text{abc}}=0.5$	6.7%
TASP-dPose (time-based)	$R_A = \gamma_{\text{sp}} \cdot \max(0, t_B - t_A)$, no ABC	16.7%

Findings:

- Time-based Alice (TASP-dPose) outperforms outcome-based (ASP-dPose) by +10pp
- TASP-dPose disables ABC – the gain comes from the time-based reward alone
- Best self-play variant (TASP-dPose, 16.7%) still **4.8×** below single-agent (PPO-PBRS, 80.0%)

[Plappert et al. 2021] [Sukhbaatar et al. 2018] [Dennis et al. 2020]

23

ASP-dPose vs TASP-dPose — Alice Reward

The two self-play variants differ only in Alice’s reward: outcome-based (ASP-dPose) vs time-based (TASP-dPose), both with the SE(2) d_{pose} Bob reward on the T-block

Time-based Alice intentionally aligns Alice’s reward with solvable goals — she gains when Bob takes more pushes than she did

Time-based reward outperforms outcome-based: TASP-dPose 16.7% vs ASP-dPose 6.7%

But even the best self-play variant (TASP-dPose, 16.7%) is 4.8× below single-agent PPO-PBRS (80.0%)

ABC is disabled in TASP-dPose — the improvement comes from time-based reward alone, not from behavioral cloning

Validation Test Suite – Top 10 Held-Out Scenes



\textbf{Top row:} 5 easiest scenes (3-4 pushes) | Start pose | Goal pose | 5 hardest scenes (0 of 4 models pass)

T-block **start pose** → **goal pose**. Every model is evaluated on an identical 30-scene suite (20 trials × up to 30 pushes per scene).

25

Validation Test Suite — Top 10 Held-Out Scenes

The full validation suite — 30 held-out T-block scenes, identical for every model

Row 1: rotation-dominant scenes (large yaw change); Row 2: position-only (goal yaw = 0); Row 3: combined position + rotation

Sky-blue = object start pose, orange = goal pose, arrow = required translation

Difficulty within each row spans easy/medium/hard by start-to-goal distance and yaw change

Isaac Lab – the Right Venue for Self-Play



Throughput (push-macros/s, measured):		
Setting	push/s	1M pushes
Isaac (1 GPU, 528 env)	172	~1.6 h
gym single-agent (6 CPU)	91	~3.0 h
gym self-play (1 CPU)	22.4	~12.4 h

Three pillars:

- **GPU-batched** – 528 envs/step gives 7,920-sample PPO batches; the self-play objective needs a large batch for stable gradients.
- **Robotic fidelity** – UR5e + 3D contact + cuRobo IK = the actual SE(2) task; gym-pusht is a 2D diagnostic.
- **Time-to-scale** – ~10M pushes in ~16h on GPU vs ~5 days on CPU.

Key result: Isaac gives self-play its best shot at a budget matched to the winning single-agent – and it **still** reaches only 6.7–16.7% (4.8× gap)

26

Isaac Lab — the Right Venue for Self-Play

The throughput table shows three tiers: Isaac GPU-batched (172 push/s, from logged TF-event timestamps: $0.022 \text{ it/s} \times 7,920 \text{ push-macros/iter}$), gym multi-core single-agent (91 push/s), gym single-core self-play (22.4 push/s)

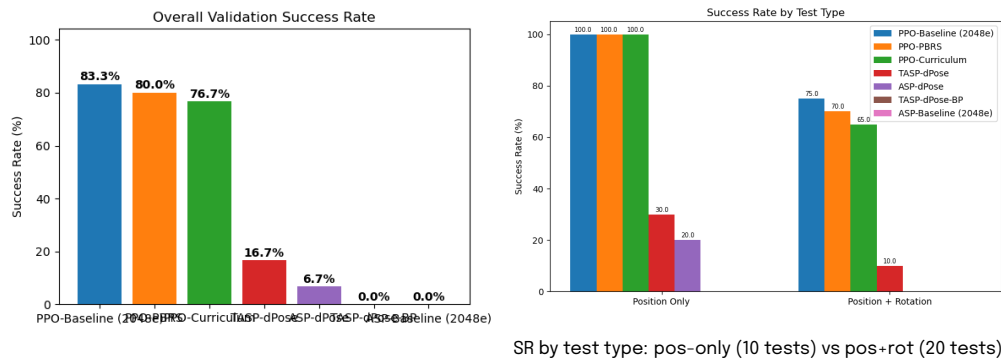
GPU-batched: 528 envs stepped in parallel → large PPO batch without extra wall-clock. The non-stationary two-agent self-play gradient needs a large batch to be stable — Isaac provides this naturally

Robotic fidelity: the real UR5e + cuRobo kinematic chain + 3D contact physics matters for the SE(2) coupled push task. gym-pusht is a useful diagnostic for the reward question, but not for the full self-play dynamics

Time-to-scale: self-play reaches its training budget (10M pushes) in 16h on Isaac vs 5 days on CPU. Self-play and single-agent have identical per-iteration timing in Isaac (1.0×). So the budget is fair

The takeaway: Isaac is not hiding self-play’s failure — it’s giving it its best shot. The 4.8× gap is real and structural, not an artifact of under-resourcing

Multi-Model Validation



Overall SR & SE – 30 T-block scenes, best-of-20
PPO-PBRS vs PPO-Curriculum: 80.0% vs 76.7% – 3.3pp gap **not statistically significant**
 ($p > 0.05$ on 30 independent scenes). **Self-play (ASP-dPose, TASP-dPose):** 4.8–11.9× below

28

Multi-Model Validation

Multi-model comparison with ± 1 standard error bars from 30 independent scenes

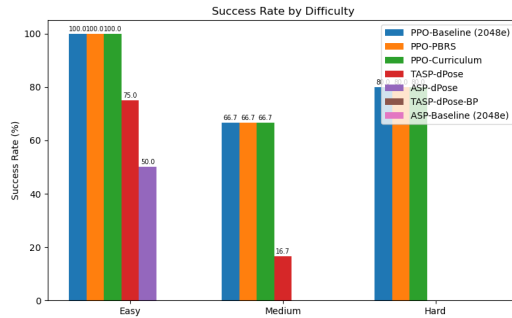
Left: overall SR with error bars. PPO-PBRS at 80.0% vs PPO-Curriculum at 76.7% — overlapping CIs. The 3.3pp gap is NOT statistically significant ($p > 0.05$, SE of difference = ± 10.6 pp). Both solve the task similarly well

Right: by test type. Both PPO-PBRS and PPO-Curriculum achieve 100% pos-only. The self-play variants barely solve any pos+rot tests — this gap IS significant

Base ASP and GoalEncoder-ablation excluded: no matched-protocol Isaac validation

Key takeaway: the error bars make clear that PPO-PBRS and PPO-Curriculum are indistinguishable, while self-play is fundamentally worse

Comparison Summary – All Models with Error Bars



Key numbers (30 T-block scenes):

Model	SR	SE	95% CI
PPO-PBRS	80.0%	7.3%	65–95%
PPO-Curriculum	76.7%	7.7%	61–92%
TASP-dPose	16.7%	6.8%	3–30%
ASP-dPose	6.7%	4.6%	0–16%
TASP-dPose-BP	0.0%	–	–

PPO-PBRS vs PPO-Curriculum gap: 3.3pp – not

statistically significant ($p > 0.05$). Both solve the task similarly. Self-play variants are 4.8–11.9× below – this gap **is** significant.

Error bars = ± 1 standard error across scenes. Base ASP and the GoalEncoder-ablation are excluded – neither has a matched-protocol Isaac validation. See thesis appendix.

29

Comparison Summary — All Models with Error Bars

Multi-model comparison with ± 1 SE error bars from 30 independent scenes

Left: SR by difficulty. PPO-PBRS and PPO-Curriculum dominate across all categories, with overlapping error bars — statistically indistinguishable. Self-play variants perform near-zero on medium/hard

Right: CI summary table. The 3.3pp gap is NOT statistically significant ($p > 0.05$, SE diff = ± 10.6 pp). Both solve the task similarly well

Self-play gap of 4.8–11.9× IS statistically significant relative to single-agent

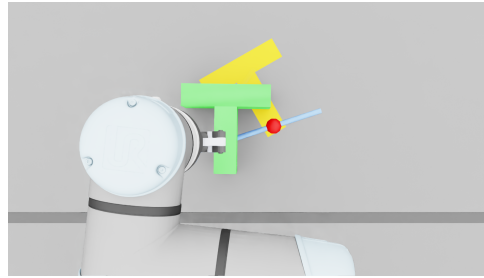
Base ASP and GoalEncoder-ablation excluded — no matched-protocol Isaac validation

PPO-PBRS – Validation Results



30 held-out T-block scenes (20 trials each):

- **80.0% SR** (24/30) – pos-only **100%**, pos+rot **70%**
- PosErr **0.032 m**, RotErr **0.568 rad** (mean over all 30 scenes, incl. failures)
- Per-attempt (trial-level) SR **66%**; scene SR counts a scene solved if any of its 20 trials passes



Overhead view – PPO-PBRS solving a pos+rot scene [▶ play clip](#)

[Ng et al. 1999] [Grzes 2017] [Schulman et al. 2017]

30

PPO-PBRS — Validation Results

Validation: 30 held-out T-block scenes, 20 trials per scene. 80.0% scene SR

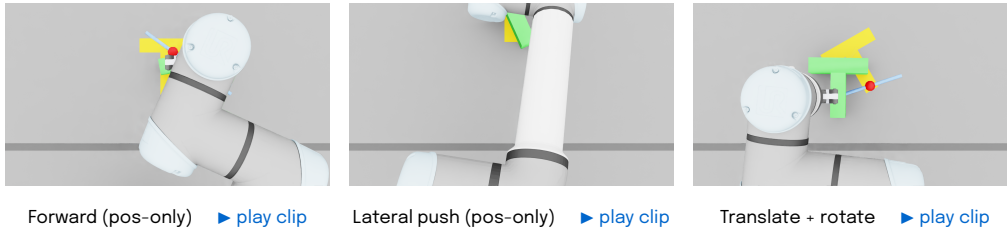
Perfect on position-only scenes (100%), strong on combined pos+rot scenes (70%)

PosErr 0.032m and RotErr 0.568 rad are averaged over all 30 scenes, including failures (RotErr is inflated by failed and by pos-only scenes whose rotation is ungated). On successful scenes PosErr $\approx$ 0.010m

Per-attempt SR is 66% (396/600 trials); the 80% scene SR counts a scene solved if any of its 20 trials passes

The overhead clip shows the trained PPO-PBRS policy recorded top-down

PPO-PBRS – Validation Runs (overhead)



35

PPO-PBRS — Validation Runs (overhead)

Self-play fails on *both* axes across almost all scenes — rotation error stays > 1.4 rad. The few successes are easy position-only scenes. Both clear position on nearly every scene; the failures are **rotation**-limited (red bars, bottom) on the hardest pos+rot scenes.

Three representative validation runs recorded top-down with `record_push_video.py`

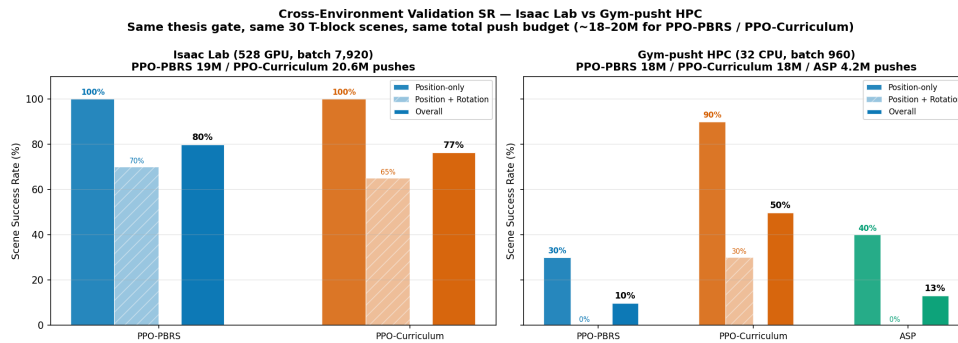
Left: a short forward push (E_Foward, position-only)

Middle: a lateral push (E_Left, position-only)

Right: a combined translate-and-rotate scene (E_Diag, position+rotation)

All use the same simple PPO-PBRS checkpoint with object-relative obs+actions

Cross-Environment Validation – Isaac vs Gym



The ordering single-agent $\approx$ curriculum $\gg$ self-play holds in both simulators. Gaps compress in the simpler gym env: PPO-Curriculum benefits most from the small-batch regime (50% gym vs 76.7% Isaac), consistent with the RQ3 batch-size caveat.

36

Cross-Environment Validation — Isaac vs Gym

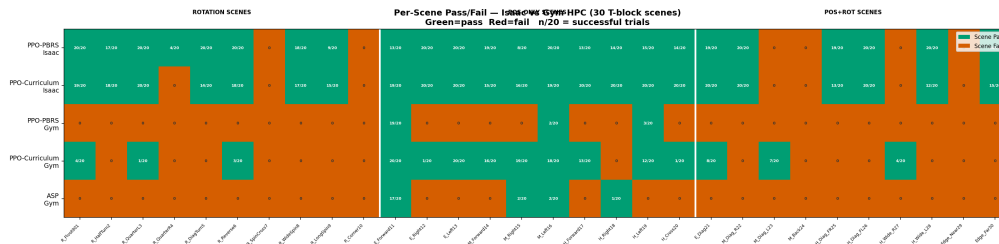
Same models, same 30-scene thesis gate, two simulators. Left: Isaac (batch 7,920). Right: gym HPC (batch 960)

PPO-PBRS and PPO-Curriculum dominate in both; self-play (ASP, gym only as a genuine run) stays low

Key nuance for RQ3: at the small gym batch, PPO-Curriculum (50%) far exceeds PPO-PBRS (10%) — curriculum helps when gradients are noisy; at the large Isaac batch they tie. The substitution is regime-dependent

There is no matched-protocol self-play Isaac best-of-20 entry, so the Isaac panel shows only the two single-agent variants

Per-Scene Outcomes – Isaac vs Gym



Green = scene solved (any of 20 trials), red = failed; cell text = successful trials / 20. The hardest pos+rot scenes (right block) defeat every configuration except the two single-agent models on Isaac. Self-play (gym) solves only a few easy position-only scenes.

[Ng et al. 1999] [Grzesiński 2017] [Plappert et al. 2021]

37

Per-Scene Outcomes — Isaac vs Gym

Per-scene pass/fail across the 30 scenes for each model×simulator

Three blocks: rotation scenes, position-only scenes, pos+rot scenes

PPO-PBRS / PPO-Curriculum on Isaac fill most of the grid; the same models in gym solve fewer; self-play (gym) only the easiest position-only scenes

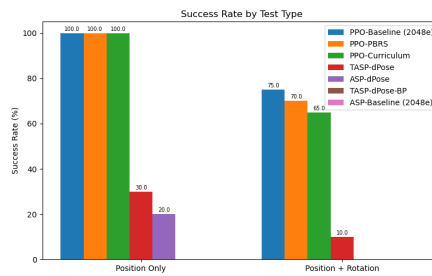
The pos+rot block is the universal bottleneck — only the Isaac single-agent models clear it consistently

Discussion – What the Validation Suite Reveals



The combined pos+rot gate is the wall:

- Position-only scenes (T11-T20): solved by both single-agent models (100%).
- Pos+rot scenes (T1-T10, T21-T30): the bottleneck – PPO-PBRS 70%, self-play $\leq 10\%$.
- Failures are almost always **rotation**-limited, not position – consistent with the coupled limit-surface physics.
- Hardest scenes are large-yaw / wide-translation (e.g. R_SpinCross, H_Wide): they need many coordinated



39

ASP Diagnostic — Alice vs. Bob

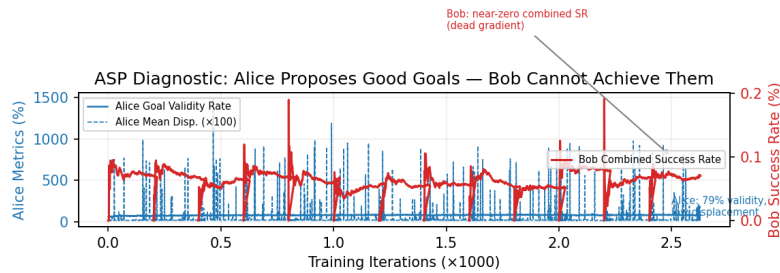
Plot shows Alice's goal validity rate rising while Bob's success rate stays near zero — the adversarial curriculum prevents combined objective achievement

Alice creates valid, low-displacement goals — she is not proposing impossible poses

Individual per-axis metrics suggested Bob partially learns position and rotation separately, but the combined gate rarely fires. This gap is likely a gate/measurement artifact (C4): per-axis metrics are not independent Bernoulli trials — the combined metric is the only valid success measure

Time-based self-play (TASP-dPose) partially rescues this: 16.7% vs 6.7% for outcome-based (ASP-dPose), suggesting the time-based Alice reward self-regulates toward solvable goals

ASP Diagnostic – Alice vs. Bob



Alice proposes valid, low-displacement goals, yet Bob cannot achieve the combined pos+rot objective under the non-stationary adversarial goal distribution.

[Plappert et al. 2021] [Sukhbaatar et al. 2018] [Campero et al. 2021]

40

Why ASP Fails — Failure Modes

Failure mode 1 — non-stationary goal distribution: Alice learns continuously, so Bob's training distribution shifts every iteration. PPO computes the importance ratio using old `logp` from a rollout collected under a different Alice. The ratio is dominated by Alice's distribution shift, not Bob's weight updates

Failure mode 2 — confounded ablation: PPO-PBRS (single-agent) and the self-play variants differ on at least 3 axes simultaneously (goal distribution, agent count, budget). The performance gap could be due to non-stationary goals rather than the self-play mechanism itself

The time-based variant (TASP-dPose) partially addresses both: self-regulating reward aligns Alice with Bob's learning progress; disabling ABC removes a third failure mode (vanishing BC gradient). TASP-dPose achieves 16.7%, 10pp above outcome-based ASP-dPose at 6.7%

Caveat – Confounded Ablation



Problem: the self-play variants differ from single-agent PPO-PBRS on *at least 4 axes simultaneously*:

Variable	PPO-PBRS (80.0%)	Self-play (6.7-16.7%)
Agent count	1	2 (Alice + Bob)
Goal distribution	Fixed random	Adversarial, shifting
GoalEncoder	None	ϕ -MLP 8D latent
ABC buffer	None	Yes (ASP-dPose) / No (TASP-dPose)
Historical pool	None	Yes
Episode budget	15 pushes/agent	5 (Alice) + 10 (Bob)
PPO updates/iter	1	2 (Alice, then Bob)

The 4.8× gap cannot be attributed to any single lever.

- TASP-dPose (time-based Alice, no ABC) vs ASP-dPose (outcome-based, ABC enabled) isolates the Alice reward axis – TASP-dPose gains +10pp
- But this is a *single* 1-axis experiment within a 7-axis package change

43

Caveat — Confounded Ablation

The self-play variants and single-agent differ on at least 4 axes — agent count, goal distribution, GoalEncoder, ABC, historical pool, episode budget, update frequency

The table shows every variable that changes. Same PPO/LSTM/PBRS/528-env budget — but 6+ other things change

TASP-dPose vs ASP-dPose is the only clean 1-axis comparison: time-based vs outcome-based Alice reward. TASP-dPose gains 10pp — so reward structure matters

But we cannot attribute the full 4.8× gap to any single variable. Non-stationary goal distribution may be the main culprit, but it's confounded with agent count and budget

The substitution claim (next slide) is about reward vs curriculum — not about single-agent vs self-play architecture

What We Found – Theory vs Practice



Self-play did *not* yield a useful curriculum in this setting

In this contact-rich, IK-gated, multi-objective SE(2) pushing task under a single-GPU budget, our self-play implementation produced 6.7–16.7% validation SR vs 80.0% for single-agent. The adversarial mechanism did not generate a learnable progression of goals. This does *not* refute self-play in general – it demonstrates a failure mode under realistic robotics constraints.

The bottleneck is the goal *distribution*, not the goal *representation*

Bob's individual position and rotation skills are partly learned, but the combined gate rarely fires under the non-stationary adversarial goal distribution. The failure sits at the exploration / training-distribution level, not the observation encoding.

Consistent across reward schemes, simulators, and ASP variants

45

Key Result — PBRs Substitutes for Curriculum

Central contribution (RQ3): a well-shaped reward can substitute for an explicit curriculum

PBRs single-agent 80.0%. Adding curriculum 76.7% — marginal, statistically indistinguishable gap ($3.3\text{pp} \pm 10.6\text{pp SE}$). So at Isaac scale the curriculum is redundant

Scope: at small PPO batch (gym, 960) the curriculum helps (50% vs 10%) — the substitution is regime-dependent, not absolute. State this honestly

Self-play (6.7–16.7%) is 4.8–11.9× worse — but recall the confound: it changes ≥ 4 axes, so this is "self-play as implemented," not "the curriculum mechanism"

Practical implication: invest in reward design first; it is the higher-leverage intervention

What We Found – Theory vs Practice



Forced curriculum: functional but not beneficial

PPO-Curriculum’s original trigger (per-push mean PosErr $< 0.08\text{m}$) was mis-specified – the threshold was unreachable by construction. A corrected trigger (episodic position-SR ≥ 0.5) produces a competent model (76.7%), but the staged pos→rot curriculum does *not* beat the simpler no-curriculum baseline (80.0%). Curriculum works – it just doesn’t add value at this batch size.

PBRs theory strongly supported

The policy invariance theorem holds in practice, even in contact-rich manipulation with stochastic PhysX physics. PBRs provides clean, bounded, Markovian rewards. Parameters $k_p=30$, $k_r=5$, $w=10.0$ are robust across all single-agent runs.

[Ng et al. 1999] [Grzes 2017] [Sukhbaatar et al. 2018]

46

What We Found — Theory vs Practice

Claim 1 (Plappert): self-play generates an intrinsic curriculum for exploration. In our specific setting — contact-rich 3D pushing with cuRobo IK gating and a strict combined pos+rot success gate — it produced adversarial rather than constructive goals. But this is N=1 task, single budget. We do not claim this generalizes

Claim 2 (Sukhbaatar): the goal-encoder bottleneck helps Bob represent the goal. Our self-play runs fail at the exploration / distribution level, not the representation level — so representation tweaks cannot rescue it here

What We Found – Theory vs Practice



Forced curriculum: functional but not beneficial

PPO-Curriculum's original trigger (per-push mean PosErr $< 0.08\text{m}$) was mis-specified – the threshold was unreachable by construction. A corrected trigger (episodic position-SR ≥ 0.5) produces a competent model (76.7%), but the staged pos→rot curriculum does *not* beat the simpler no-curriculum baseline (80.0%). Curriculum works – it just doesn't add value at this batch size.

PBRs theory strongly supported

The policy invariance theorem holds in practice, even in contact-rich manipulation with stochastic PhysX physics. PBRs provides clean, bounded, Markovian rewards. Parameters $k_p=30$, $k_r=5$, $w=10.0$ are robust across all single-agent runs.

[Ng et al. 1999] [Grzes 2017] [Sukhbaatar et al. 2018]

46

What We Found — Theory vs Practice

Claim 3 (Narvekar/Portelas): curriculum learning helps when the base reward is informative. PPO-Curriculum's first version had a mis-specified trigger (P82 bug). Fixing it produced a functional curriculum, but with marginal degradation (76.7% vs 80.0%) — evidence that a good reward (PBRs) can substitute for the curriculum at this batch size

The PBRs theory (Ng, Grzes) is strongly supported. The policy invariance theorem produced robust results — the same hyperparameters work across independent training runs

This speaks to RQ3: a sufficiently informative reward substitutes for an explicit curriculum (at sufficient batch size)

Conclusion



1. PBRS enables on-policy learning for planar pushing (RQ1)

PPO with the original fractional reward requires 2,048 envs to reach 80%. PBRS achieves identical performance at 528 envs – $4\times$ fewer – by providing dense, correctly-signed, bounded rewards on *every* push. PBRS preserves the optimal policy while making the reward tractable for on-policy GAE.

2. Self-play fails structurally on this specific task (RQ2)

Across five ASP variants – original fractional reward (0%), outcome-based PBRS (6.7%), time-based PBRS (16.7%), symmetric Bob-penalty PBRS (0%), and a gym-pusht implementation (13.3%) – **no self-play configuration exceeds 17%**, against an 80% single-agent ceiling. The adversarial goal distribution creates a perpetually stale value function that prevents stable on-policy gradient estimates.

48

Conclusion

Take-home: PBRS works. PPO-PBRS 80.0% on 30 held-out T-block scenes with the simplest architecture. Theory (policy invariance, episodic validity, bounded potentials) supported by empirical results

Self-play performs worse than single-agent in this setting. Time-based Alice partially rescues (16.7% vs 6.7%) but the $4.8\times$ gap remains. Self-play & single-agent run at identical per-iteration wall-clock — the gap is structural, not a budget artifact

RQ3: a well-shaped reward substitutes for a curriculum at sufficient batch size. Future work: (a) design good rewards (PBRS), (b) diagnose the training distribution, (c) add complexity only with evidence

Practical recommendation: simplest working system first. 528-env PPO-PBRS trains in ~ 1.6 days on one RTX Pro 6000, achieves 80.0% SR

Limitations & Research Gaps



What we couldn't test (training budget limited to 3000 iters $\times$ 528 envs):

- **TASP-dPose-BobPenalty tested:** Bob time penalty (full Sukhbaatar symmetric reward) collapsed to 0% – penalty applied on catastrophe phase ends created a perverse “fail-fast” equilibrium. Asymmetric reward structure (Alice incentivised, Bob unpenalised) proved necessary.
- **Time-based self-play + ABC:** time-based Alice with ABC enabled ($\beta_{abc} = 0.5$). ABC might boot after Alice converges
- Larger models (Transformer, diffusion policy), sim-to-real transfer (physical UR5e)

Remaining literature gaps (from systematic survey of 81 papers):

- No paper on SE(2) metric design specifically for RL reward shaping
- No paper on exploiting continuous rotational symmetry (SO(2)) in object manipulation RL
- No formal convergence analysis of effort-based self-play rewards

50

Future Work — Promising Directions

Results are conclusive within the experimental budget; self-play directions remain open

TASP-dPose-BobPenalty: Bob time penalty creates near-zero-sum game structure. From Letcher et al. (2019) zero-sum games have convergent (Hamiltonian) dynamics — could stabilise self-play where general-sum diverges

ABC after Alice converges ($\beta=0.5$) — currently ABC fails because Alice's actions are random ($bc_ratio \approx 1.0$)

Relaxed success gate: per-axis SR is high but combined gate rarely fires. Partial credit could provide gradient toward both objectives

PAIRED: antagonist minimises protagonist's regret rather than maximising difficulty — solvable but challenging environments

8. Conclusion & Takeaways

Computational Overhead — Self-Play Is Not Slower

Controlled, same-machine measurement: self-play and single-agent run at identical iteration cost (0.022 it/s) on the same 528-env GPU setup. The 2-agent machinery adds no per-iteration wall-clock because cuRobo IK and physics dominate

I deliberately do NOT claim a 5–10× self-play overhead — there is no controlled measurement supporting that; the only honest number is 1.0× per iteration

Key distinction: self-play doesn't fail because it's computationally expensive per iteration. It fails because it doesn't learn. The two are separate problems

The measured Isaac-vs-gym throughput (172 push/s vs 22 push/s for self-play on CPU) is shown in the Isaac Lab venue slide (Evaluation) — establishing that Isaac Lab is the right venue, not a handicap

Extra Takeaway – ManagerBasedRL vs DirectRL



An engineering observation, separate from the scientific claims.

- Isaac Lab's **ManagerBasedRL**Env (Action/Observation/Event/Termination managers) adds per-step Python dispatch; **DirectRL**Env bypasses it with direct tensor ops.
- This is an *architecture* cost, independent of ASP – self-play and single-agent run at the same per-iteration time ($\sim 1.0\times$).

API	env-steps/s	relative
DirectRL	[to measure]	[run pending]
ManagerBasedRL	[to measure]	[run pending]

Controlled same-task measurement to be run; numbers will replace the placeholders. No multiplier is claimed until then.

[Makoviychuk et al. 2021] [Rudin et al. 2022]

53

PBRS — The Shaping Theorem

The fundamental theorem: add $F = \gamma \cdot \Phi(s') - \Phi(s)$ without changing optimal policies

Proof sketch: $Q'(s,a) = Q(s,a) - \Phi(s)$. Since $\Phi(s)$ depends on state only (not action), $\arg\max$ unchanged

Our potential: sum of two exponentiated distance functions. Cosine $c(s)$ respects S^1 topology — no discontinuity at $\pm\pi$

Episodic PBRS: $\gamma_{\text{shaping}}=1.0$ is valid — sum over an episode is bounded = $\Phi(s_T) - \Phi(s_0)$



university of
 groningen

Thank You – Questions?

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026

PBRS — Why It's the Right Choice

Old formula violated this by dividing by d_{prev} .

Four properties that make PBRS ideal for contact-rich pushing: Markovian (depends only on s, s'), Bounded (exponentials cap reward range), Zero-sum cycles (detour pushes give zero net F — no reward hacking), No constant drain (stationary agent gets $F=0$)

The old fractional improvement formula violated all four — non-Markov (divides by d_{prev}), unbounded (division by 0.01), negative-sum (staying still was penalised)

Appendix Outline



- A. Push Mechanics – Limit Surface & Characteristic Length L
- B. Training curves for all PBRS models (Success Rate, Reward, Loss)
- C. PBRS parameter sensitivity (k_p , k_r , w_{pos} , w_{rot})
- D. Validation scene descriptions (30 test cases)
- E. cuRobo IK pipeline and workspace configuration
- F. Observation space specifications (all model variants)
- G. Per-test error bars – PPO-PBRS vs PPO-Curriculum head-to-head and all-model comparison (95% binomial CIs)

55

PPO-PBRS — Training Dynamics

Left plot: training success rate over 3000 iterations. PPO-PBRS (blue) improves from near-zero to 9.4% (strict combined gate). PPO-Curriculum (orange, P82-fixed) follows a similar trajectory. Self-play variants remain near baseline

Right plot: value loss stays stable — no explosion. The PBRS exponential potentials prevent the reward spikes that caused GAE chain reactions and critic instability in earlier attempts

PPO-Curriculum – Position→Rotation Curriculum

Design: Same PPO agent as PPO-PBRS + 3-phase curriculum:

1. $w_{\text{rot}} = 0$, position-only training
2. Triggered when episodic position-SR ≥ 0.5 for 20 iters: w_{rot} ramps $0 \rightarrow 10$ over 200 iters
3. Full PBRS (multi-objective)

Original trigger was mis-specified: the first version used a per-push mean position error threshold of 0.08m – below even the *best* model's validation PosErr (0.202m). The curriculum never activated because the threshold was unreachable by construction (P82 fix).

Corrected trigger: gates on episodic position-only SR ≥ 0.5 over a 20-iter lookback. Curriculum now activates and reaches **76.7% SR** – only 3.3pp below PPO-PBRS.

[Narvekar et al. 2020] [Portelas et al. 2020] [Florensa et al. 2017]

20

PPO-Curriculum vs PPO-PBRS

Head-to-head on identical 30 T-block scenes: PPO-PBRS 80.0% vs PPO-Curriculum 76.7%

PPO-Curriculum actually beats PPO-PBRS on position precision (0.023 vs 0.032 m) — the pos-only Phase 1 training produces tighter position control. But rotation is slightly worse (0.663 vs 0.568 rad)

Both achieve 100% pos-only SR. The combined pos+rot gate is 70% (PPO-PBRS) vs 65% (PPO-Curriculum)

The curriculum is functional but fails to outperform the simpler baseline. Adding structure adds complexity without adding net value

PBRS – Why It’s the Right Choice



Properties of $F(s, s') = \Phi(s') - \Phi(s)$:

1. **Markovian** – depends only on s, s' , not history.
2. **Bounded** – exponential potentials prevent reward explosions from PhysX contact spikes.
3. **Zero-sum cycles** – detour pushes net to zero; agent free to explore without reward hacking.
4. **No constant drain** – $F=0$ when stationary; no penalty for hesitation or deliberation.

Key consequence: you can replace any ad-hoc reward with a potential-based one *without changing what the agent learns* – only *how fast* it learns. Reward design and policy optimisation are decoupled.

[Ng, Harada & Russell 1999] [Grzes 2017] [Harutyunyan et al. 2015]

57

Self-Play — Validation Comparison

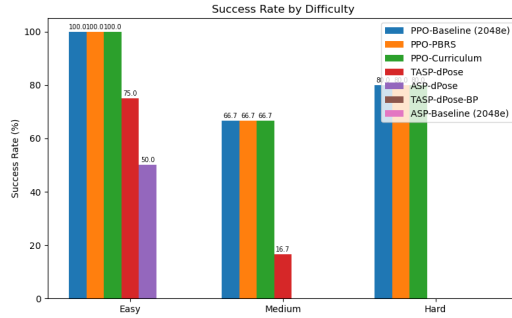
PPO-PBRS (single-agent, reference) at 80.0% vs TASP-dPose (best self-play variant) at 16.7% — 4.8× gap

Time-based Alice (TASP-dPose) partially corrects the adversarial curriculum by rewarding solvable goals — gains 10pp over ASP-dPose

But even the best self-play variant cannot close the gap. The self-play dynamics prevent learning the combined pos+rot objective

Both self-play variants trained identically (528 envs, 3000 iters, PBRS dense reward for Bob) — the only difference is Alice’s reward structure

Comparison Summary – All Models with Error Bars



Key numbers (30 T-block scenes):

Model	SR	SE	95% CI
PPO-PBRS	80.0%	7.3%	65–95%
PPO-Curriculum	76.7%	7.7%	61–92%
TASP-dPose	16.7%	6.8%	3–30%
ASP-dPose	6.7%	4.6%	0–16%
TASP-dPose-BP	0.0%	–	–

PPO-PBRS vs PPO-Curriculum gap: 3.3pp – not

statistically significant ($p > 0.05$). Both solve the task similarly. Self-play variants are 4.8–11.9× below – this gap **is** significant.

Error bars = ± 1 standard error across scenes. Base ASP and the GoalEncoder-ablation are excluded – neither has a matched-protocol Isaac validation. See thesis appendix.

29

Comparison Summary — Per-Scene Position $\times$ Rotation Error

Three-panel scatter: each point is one held-out scene — position error vs rotation error; the green box is the success region (pos < 5 cm, rot < 0.2 rad). PPO-PBRS fills it; PPO-Curriculum clusters position inside it but rotation outside it; ASP-dPose scatters far out.

The curriculum forces consistent position control from Phase-1 but cannot fix the combined pos+rot gate; self-play fails both axes.